

MODULE2

topics: Functions, Lambda Functions, List Comprehensions, File Handling, CSV, JSON, Virtual Environments,

Exception Handling, Debugging and Student Record Management System.

FUNCTIONS

A **function** is a block of code used to perform a specific task.

Instead of writing the same code again and again, we create a function and **call it whenever needed**.

Basic Syntax

```
def function_name():
```

Example:

```
def greet():  
    print("Hello Ammu")
```

```
greet()
```

Output:

```
Hello Ammu
```

Function with Parameters

Parameters are values we give to a function.

Example:-

```
def greet(name):  
    print("Hello", name)
```

```
greet("Ammu")
```

Output:

```
Hello Ammu
```

Function with Return Value

return sends a value back from the function.

```
def add(a, b):  
    return a + b
```

```
result = add(10, 20)
print(result)
```

Output:

30

LAMBDA & LIST COMPREHENSION

1. Lambda functions
2. List comprehensions

1. Lambda Function

A lambda is a **small one-line function**.

Example:

```
square = lambda x: x * x

print(square(5))
```

Output:

25

List Comprehension

List comprehension is a **short way to create a list**.

Example:

```
numbers = [x for x in range(1, 6)]

print(numbers)
```

Output:

[1, 2, 3, 4, 5]

Example – Even Numbers

```
even = [x for x in range(1, 11) if x % 2 == 0]

print(even)
```

Output:

[2, 4, 6, 8, 10]

FILE HANDLING, CSV & JSON

This is an important part because now you learn how Python can **store information in files**.

You learn:

1. File handling
2. CSV
3. JSON

1. File Handling

Python can create, write, read and add information to files.

Write to a file

```
file = open("student.txt", "w")
```

```
file.write("Name: Ammu")
```

```
file.close()
```

This creates:

student.txt

and stores:

Name: Ammu

Read a File

```
file = open("student.txt", "r")
```

```
data = file.read()
```

```
print(data)
```

```
file.close()
```

Output:

Name: Honey

with open()

A better way to work with files is:

with open("student.txt", "r") as file:

```
    data = file.read()
```

```
print(data)
```

CSV

CSV means:

Comma-Separated Values

Example:

ID,Name,Course

101,Honey,AIML

102,jasu,CSE

CSV is commonly used for storing **table-like data**.

Student ID | Name | Course

101 | Pujitha | AIML

102 | Ravi | CSE

JSON

JSON is another format for storing data.

Example:

```
student = {
```

```
"name": "Pujitha",  
"age": 20,  
"course": "AIML"  
}
```

EXCEPTION HANDLING & DEBUGGING

Day 9 teaches you how to handle errors safely.

Topics:

- Errors/exceptions
- `try`
- `except`
- `finally`
- Virtual environments
- Debugging

What is an Exception

An exception is an error that happens while the program is running.

For example:

```
a = 10  
b = 0  
  
print(a / b)
```

This gives a:

```
ZeroDivisionError
```

because we cannot divide by zero.

try-except

We can handle the error using `try` and `except`.

`try:`

```
a = 10
```

```
b = 0
```

```
print(a / b)
```

`except ZeroDivisionError:`

```
print("Cannot divide by zero")
```

Output:

Cannot divide by zero

finally

`finally` runs whether there is an error or not.

`try:`

```
print("Hello")
```

`except:`

```
print("Error")
```

`finally:`

```
print("Program completed")
```

Output:

Hello

Program completed

ValueError

A `ValueError` can happen when we give an invalid value.

Example:

try:

```
age = int(input("Enter age: "))
```

```
print(age)
```

except ValueError:

```
print("Please enter a number")
```

If the user enters:

abc

Output:

Please enter a number

Debugging

Debugging means finding and fixing errors in a program.

Example:

```
a = 10
```

```
b = 20
```

```
print(a + b)
```

MINI PROJECT

```
students = []
```

```
while True:
```

```
    print("\n--- Student Record System ---")
```

```
    print("1. Add Student")
```

```
    print("2. View Students")
```

```
    print("3. Search Student")
```

```
    print("4. Exit")
```

```
choice = input("Enter your choice: ")

if choice == "1":

    name = input("Enter student name: ")

    age = input("Enter student age: ")

    course = input("Enter course: ")

    student = {

        "name": name,

        "age": age,

        "course": course

    }

    students.append(student)

    print("Student added successfully!")

elif choice == "2":

    if len(students) == 0:

        print("No students found.")

    else:

        for student in students:

            print(student)

elif choice == "3":

    name = input("Enter student name to search: ")

    found = False

    for student in students:

        if student["name"] == name:

            print("Student found:")

            print(student)

            found = True
```

```
    if found == False:

        print("Student not found.")

elif choice == "4":

    print("Thank you!")

    break

else:

    print("Invalid choice.")
```